

SDE-II Backend Architecture Assignment

Assignment: Design an AI-Powered Project Evaluation Platform

Time Allowed

48 hours from the time the assignment is received.

Submission

Link -

https://forms.gle/W2unJoASEuoFamXE8

Objective

AlmaBetter evaluates student projects as part of its learning and assessment process.

We want to build a platform where a student submits a project, and an AI-powered evaluation system automatically analyses the submission against a predefined rubric and produces a structured evaluation.

The system should eventually support thousands of submissions and different types of projects.

Your task is to **design the backend architecture and technical solution for this system**.

You are **not expected to build the complete application**.

We are primarily evaluating your ability to:

- Think about backend architecture
- Design scalable APIs and data models
- Handle asynchronous workloads
- Integrate LLMs into production systems
- Design for reliability and failure recovery
- Make appropriate technology choices
- Identify security and scalability concerns
- Think beyond the happy path

The role requires experience across APIs, databases, scalable architecture, AI integrations, cloud deployment and production systems.

Business Scenario

A student submits a project through the platform.

A submission may contain one or more of the following:

- GitHub repository
- ZIP file
- PDF
- Google Drive folder
- Video demonstration

- Documentation
- Live application URL

The platform should then:

```
Student Submission
        ↓
Submission Processing
        ↓
File / Repository Extraction
        ↓
Project Understanding
        ↓
AI Evaluation
        ↓
Rubric-Based Scoring
        ↓
Human Review (if required)
        ↓
Final Evaluation
        ↓
Student Dashboard
```

The AI should evaluate the project against a rubric such as:

Dimension	Weight
Functionality	25%
Code Quality	20%
Architecture	20%
Problem Solving	15%
Documentation	10%
Innovation / AI Usage	10%

The rubric should **not be hard-coded into the evaluation engine**. Different courses and assignments may have different rubrics.

Scale Assumptions

Design the system initially for:

- **30 submissions/day**
- Average submission size: **100 MB**
- Some submissions may contain **1 GB+ repositories/files**
- Evaluation can take anywhere from **2–15 minutes**
- Multiple LLM calls may be required for a single evaluation
- Some evaluations may fail midway
- An evaluator may need to manually review an AI-generated evaluation

But the architecture should explain how you would evolve the system to:

> **10,000 submissions/day**

You don't need to over-engineer the initial system. Explain **what you would build today and what you would change as scale increases.**

Your Task

Design the backend system for the above platform.

1. System Architecture

Create a high-level architecture showing:

- Client/application layer
- API layer
- Authentication/authorization
- Backend services
- Database
- Object/file storage
- Queue/event system
- Background workers
- AI/LLM layer
- Evaluation engine
- Monitoring/logging
- External integrations

Explain **why you selected each component**.

You may choose technologies such as:

- Node.js / Python
- MongoDB / PostgreSQL
- Redis
- SQS / RabbitMQ / Kafka
- S3
- OpenAI / AWS Bedrock / open-source LLMs
- Vector database
- Docker
- AWS
- etc.

There is no prescribed technology stack.

Research the available alternatives and justify your decisions.

---

2. Submission Processing Pipeline

Design the complete lifecycle of a submission.

For example:

```
POST /submissions
       ↓
Validate submission
       ↓
Create submission record
       ↓
Upload/store files
       ↓
Create evaluation job
       ↓
Queue
       ↓
Worker
       ↓
Extract project
       ↓
Analyse files
       ↓
LLM evaluation
       ↓
Persist results
       ↓
Notify user
```

Explain:

- Which operations are synchronous?
- Which operations are asynchronous?

- Where are queues required?
- How do workers communicate?
- How do you track evaluation status?
- What happens if a worker crashes?
- What happens if an LLM API fails?
- What happens if the same submission is processed twice?

---

3. API Design

Design the key REST APIs.

At minimum, cover:

```
Create Submission
Get Submission
Upload Submission
Start Evaluation
Get Evaluation Status
Get Evaluation Result
Update Rubric
Request Human Review
Approve / Reject Evaluation
```

For each important API, provide:

- HTTP method
- Endpoint
- Request structure
- Response structure
- Authentication requirements
- Important validation rules

You don't need to implement the APIs.

---

4. Database Design

Design the database schema.

At minimum, model:

- Users
- Courses
- Assignments
- Rubrics
- Rubric Criteria
- Submissions
- Evaluation Jobs
- Evaluation Results
- Evaluation Attempts
- Human Reviews

Explain:

- Why you selected SQL or NoSQL
- Important indexes
- Relationships
- How evaluation history will be maintained
- How you would prevent inconsistent states

A simple ER diagram or collection/schema diagram is expected.

---

5. AI / LLM Architecture

This is an important part of the assignment.

A project could contain:

```
10,000 lines of code
20 PDFs
50 documentation files
Multiple images
Database schemas
API documentation
README
Video demonstration
```

You cannot simply send the entire project to an LLM in one request.

Design an approach for:

Project Understanding

How would you:

- Extract the project?
- Identify relevant files?
- Ignore irrelevant files?
- Chunk content?
- Create summaries?
- Build project context?
- Use embeddings/vector search if required?
- Prevent context-window problems?

Evaluation

How would you structure the LLM evaluation?

For example:

```
Project
   ↓
Project Understanding
   ↓
Architecture Analysis
   ↓
Code Analysis
   ↓
Documentation Analysis
   ↓
Rubric Evaluation
   ↓
Score Aggregation
   ↓
Final Evaluation
```

Explain whether you would use:

- One LLM call
- Multiple specialized calls
- Agents
- RAG
- Structured outputs
- Deterministic rules + LLM
- Human-in-the-loop

And **why**.

The JD specifically calls for experience with LLM APIs, RAG, AI agents/workflows and vector

databases.

---

6. Reliability & Failure Handling

Assume production failures will happen.

Explain how your system handles:

LLM failure

```
LLM API timeout
LLM rate limit
LLM unavailable
Malformed response
Token limit exceeded
```

Processing failure

```
Corrupt ZIP
Invalid GitHub repository
Large repository
Unsupported file
Worker crashes
Network failure
```

Duplicate processing

What happens if:

```
POST /evaluate
```

is accidentally called five times?

How would you implement **idempotency**?

---

7. Security

Identify the major security risks.

Particularly consider that students are uploading potentially untrusted code/files.

Discuss:

- Authentication
- RBAC
- File upload security
- Malware risk
- ZIP bombs
- Path traversal
- SSRF
- GitHub repository access
- Secrets/API keys
- LLM prompt injection
- Student data isolation
- Access to evaluation results
- Rate limiting

For each major risk, describe your mitigation.

---

8. Scaling

The initial system handles:

> 30 submissions/day

Now assume it grows to:

> 10,000 submissions/day.

Explain what changes.

Consider:

- API scaling
- Worker scaling
- Queue architecture
- Database scaling
- Object storage
- LLM throughput
- Rate limits
- Cost
- Caching
- Monitoring
- Backpressure

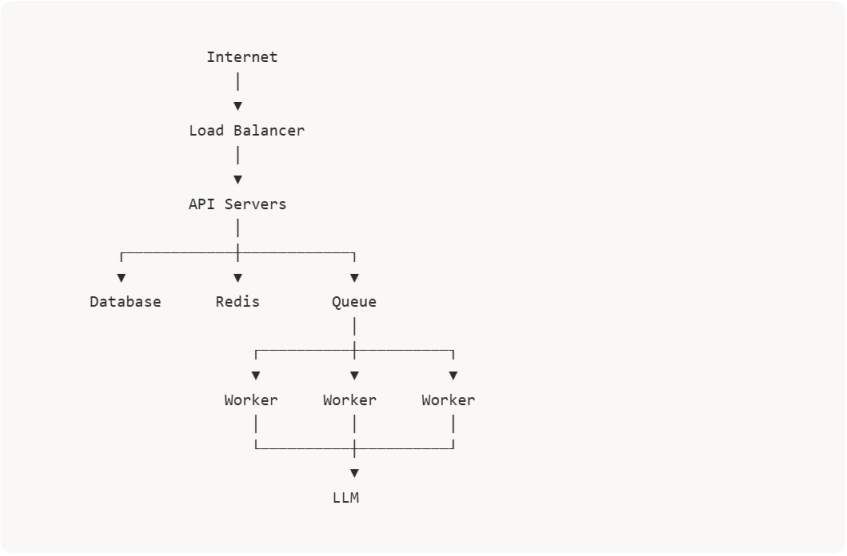
We are particularly interested in **how your architecture evolves rather than simply adding "Kubernetes" everywhere.**

---

9. Deployment Architecture

Propose a production deployment.

For example:



Explain:

- Docker
- CI/CD
- Environment management
- Secrets
- AWS services
- Logging
- Monitoring
- Health checks
- Rollbacks

- Rollbacks

The JD specifically expects exposure to AWS EC2, Docker, CI/CD and NGINX deployment.

---

10. Research Component

This is deliberately a **research assignment**.

You are expected to research relevant technologies and document your findings.

For at least **5 important technical decisions**, provide:

Decision	Options Considered	Your Choice	Reason
Database	PostgreSQL vs MongoDB	—	—
Queue	SQS vs RabbitMQ vs Kafka	—	—
LLM	API vs self-hosted	—	—
Vector DB	pgvector vs Pinecone vs Weaviate	—	—
Compute	EC2 vs containers/serverless	—	—

You should provide links to the technical documentation/articles used for your research.

We are **not looking for the "correct" technology**. We want to understand your engineering reasoning.

Deliverables

Submit **one document**, preferably PDF/Markdown, containing:

Required

1. **Architecture diagram**
2. **System design explanation**
3. **API design**
4. **Database/schema design**
5. **AI/LLM architecture**
6. **Async processing architecture**
7. **Failure-handling strategy**
8. **Security considerations**
9. **Scaling strategy**
10. **Deployment architecture**
11. **Technology decisions + alternatives considered**
12. **Implementation roadmap**

Optional

You may additionally submit:

- Small proof-of-concept
- Sample API implementation
- GitHub repository
- Docker setup
- Sample LLM evaluation pipeline

The optional implementation will not compensate for weak architectural reasoning.

Constraints

Time

Maximum: 48 hours.

Do not spend the entire time building a working application.

AI Tools

You may use:

- ChatGPT
- Claude
- Gemini
- GitHub Copilot
- Cursor
- Other AI development/research tools

However, clearly mention **where AI tools were used**.

We will discuss your architectural decisions during the interview.

External Research

You are encouraged to use:

- Official documentation
- Engineering blogs
- Architecture papers
- Cloud documentation
- Open-source project documentation

Cite the important sources you used.

What We Will Evaluate

We are primarily evaluating the following:

Area	What we want to understand
Architecture	Can you design a production backend rather than just an API?
Backend Engineering	API boundaries, services, data modelling, concurrency
Scalability	Can the system evolve from 30 → 10,000 submissions/day?
AI Architecture	Do you understand how LLM systems behave in production?
Reliability	Failure recovery, retries, idempotency, observability
Security	Can you identify risks beyond authentication?
Database	Can you make appropriate persistence and indexing decisions?
Cloud/DevOps	Can you translate architecture into deployable infrastructure?
Trade-offs	Can you explain why you chose one approach over another?
Product Thinking	Does the architecture solve the actual business problem?